Monitoring and Understanding Notebook-Based Learning with Jupyter

Arturo Olivares Martos
Universidad de Granada
Granada, Spain
Universität Duisburg-Essen
Duisburg, Germany
arturoolivares@correo.ugr.es
arturo.olivares@stud.uni-due.de

Lisa Pawlowski
Universität Duisburg-Essen
Duisburg, Germany
lisa.pawlowski.99@stud.uni-due.de

Mohamed Abdelmagied
Universität Duisburg-Essen
Duisburg, Germany
mohamed.abdelmagied@stud.uni-due.de

Kaimao Sheng
Universität Duisburg-Essen
Duisburg, Germany
kaimao.sheng@uni-due.de

Irene-Angelica Chounta
Universität Duisburg-Essen
Duisburg, Germany
irene-angelica.chounta@uni-due.de

Abstract

As programming becomes increasingly integrated into scientific and educational workflows, the need to understand how users interact with computational notebooks has grown. This project introduces a JupyterLab dashboard extension that monitors and visualizes notebook activity, including notebook openings, cell execution outcomes, errors, and retries. The system produces summary statistics and visual timelines that portray how the workflow unfolds. The dashboard is intended to be used by students for self-assessment and awareness of their coding patterns. A survey was conducted to gather user feedback on the dashboard's usability and usefulness. The insights gained from this evaluation serve as a pilot study, establishing a baseline for the tool's usability and paving the way for future research on its pedagogical impact.

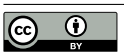
CCS Concepts

• **Applied computing** → **Interactive learning environments**; • **Human-centered computing** → **Empirical studies in visualization**.

Keywords

Jupyter Notebooks, Learning Analytics, Educational Dashboards, Student Self-Assessment, Instructor Insights, Interactive Coding Environments

ACM Reference Format:
Arturo Olivares Martos, Lisa Pawlowski, Mohamed Abdelmagied, Kaimao Sheng, and Irene-Angelica Chounta. 2026. Monitoring and Understanding Notebook-Based Learning with Jupyter. In *Proceedings of Mensch und Computer 2026 – Tagungsband, Gesellschaft für Informatik e.V. (MuC'26).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.18420/muc2026-mci-src-251



MuC'26, Duisburg, Germany

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.18420/muc2026-mci-src-251

1 Introduction

This paper presents the design of a newly developed JupyterLab extension – a learner-facing dashboard that tracks detailed notebook interactions, such as cell-level execution histories and error patterns, to support self-reflection in programming education. Understanding the programming process is important because computational notebooks are central to modern data science education, yet instructors often lack visibility into how students work, and learners themselves rarely have opportunities to reflect on their own workflows, error patterns, or problem-solving behaviors [10, 14]. Related research explores the use of learning dashboards to enhance students' self-awareness, foster reflection, and offer instructors actionable insights into learner behavior. However, the majority of existing dashboards focus on high-level learning management system data rather than fine-grained programming behavior within computational notebooks [9]. To address this gap, we develop a student-centered dashboard extension in JupyterLab and evaluate its usability, perceived usefulness, and cognitive workload through an exploratory study with 22 students using standardized evaluation instruments (SUS, TAM, and NASA-TLX) [4, 5, 7]. In particular, we aim to answer the following research question: *Whether the proposed dashboard extension is perceived by students as useful, easy to integrate into their workflow, and usable without imposing an excessive cognitive workload during programming tasks.*

This work has two contributions: 1) the design and implementation of a dashboard that captures programming behavior directly within JupyterLab, shifting focus from final code correctness to the learning process, and 2) empirical evidence on its acceptance, usability and cognitive impact, offering practical insights for building reflective tools in computational learning environments.

2 Related Work

Jupyter notebooks are widely used for scientific computing, data analysis, and programming education [10]. Their combination of executable code, narrative text, and visual output supports exploratory and iterative work practices in which learners repeatedly test, revise, and interpret code. At the same time, programming learners often struggle with error identification and debugging, frequently relying

on trial-and-error strategies [14]. Prior work on notebook-based analytics shows that cell-level traces such as execution patterns, time on task, and navigation behavior can reveal retries, hesitation, and fragmented workflows that remain invisible in final notebook submissions [1, 17]. Recent approaches to notebook analytics differ in their intended audience and purpose. Some systems provide infrastructures for collecting, analyzing, and visualizing distributed student activity, primarily to support instructors [1]. Other approaches analyze graded notebooks at assignment or course level to provide insights into performance and diversity [6], combine fine-grained notebook traces with AI-supported tutoring [17], or examine notebook changes retrospectively through repository histories [13]. More broadly, learner-facing dashboards must support students in interpreting visualizations and translating them into meaningful actions [8], while their information design may also affect cognitive load [2]. Compared with these approaches, our work focuses on embedding the system directly in JupyterLab and presenting live cell-level execution, error, retry, and focus-time information to students. The contribution therefore lies in a non-AI, learner-facing reflection interface and an initial learner-centered evaluation of its usability, perceived usefulness, ease of use, and workload.

Learning dashboards have been discussed as tools for supporting awareness, reflection, and self-regulated learning [3, 15]. However, many existing dashboards rely on coarse-grained learning management system data, such as resource access, assignment completion, or overall activity patterns, which limits their ability to represent the step-by-step nature of programming work [9]. Existing approaches in programming education often remain instructor-facing, assignment-level, or performance-oriented, rather than supporting student-directed reflection on fine-grained coding behavior [6, 16]. For learner-facing dashboards, the central challenge is not only to visualize activity data, but to support students' sense-making and actionability. [8] show that students' interpretation of dashboard visualizations depends on factors such as transparency of design, reference frames, and support for action. In addition, dashboard information design may affect cognitive load and learning performance, making workload an important evaluation dimension [2].

3 Dashboard Design and Implementation

Our learner-facing dashboard extension is designed to help students reflect on their work and identify learning gaps during programming tasks within JupyterLab. A video demo of our extension can be found here, and the data flow architecture is represented in Figure 1.

3.1 Data Collection Mechanism

Using JupyterLab's API [11, 12], the tool monitors active sessions and logs telemetry events in a relational SQLite database containing six core tables:

Users Stores anonymous personal codes and roles (student/teacher[1]).
Notebooks Tracks notebook metadata (title, path, author).
Notebook Sessions Captures timestamps for when notebooks are opened and closed and by whom.

[1]The teacher role is there for potential future use.

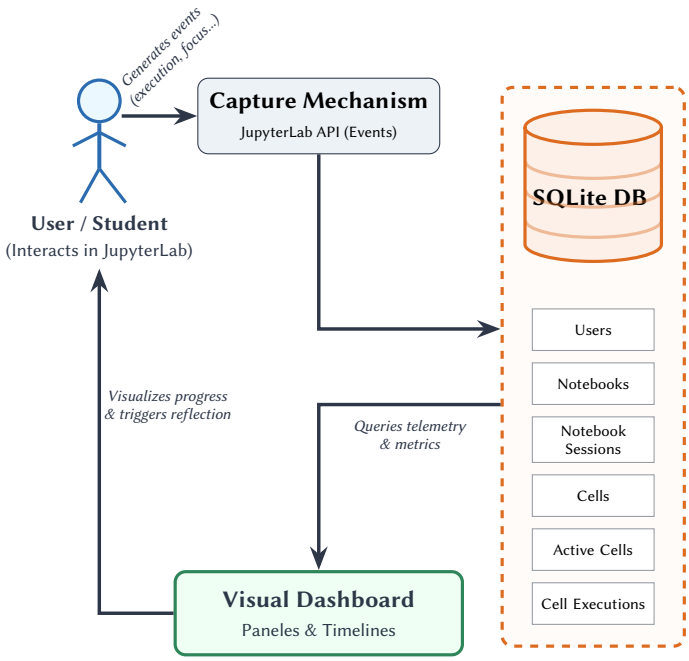


Figure 1: Data flow architecture of the learner-facing dashboard: telemetry events are captured from user interactions via the JupyterLab API, persisted into the SQLite relational database, and queried to render reflective visualizations.

Cells Logs cell types (code, markdown, raw) and original content.
Active Cells Monitors active user focus with exact interaction durations.
Cell Executions Tracks code execution metrics, runtime outputs, and error counts/messages.

3.2 Data Visualization Panels

The user interface (Figures 2 and 3), accessible through the JupyterLab command palette, is organized into distinct panels[2] (each featuring a "Help" button for interpretation):

User Information Displays the user's anonymous ID code and role.
Current Progress Shows a percentage metric tracking error-free code execution according to Equation (1).

$$\text{Progress} = \frac{\text{Code cells with an execution without errors}}{\text{Total number of code cells}} \times 100\% \quad (1)$$

Success/Error/Attempt Rate Classifies cells into three status categories and shows the rate of each category:
- *Successful*: Single execution, zero errors.
- *Attempted*: Multiple executions, but the final attempt was successful.
- *Error*: The final execution resulted in a failure.

[2]The dashboard visualization is available here.

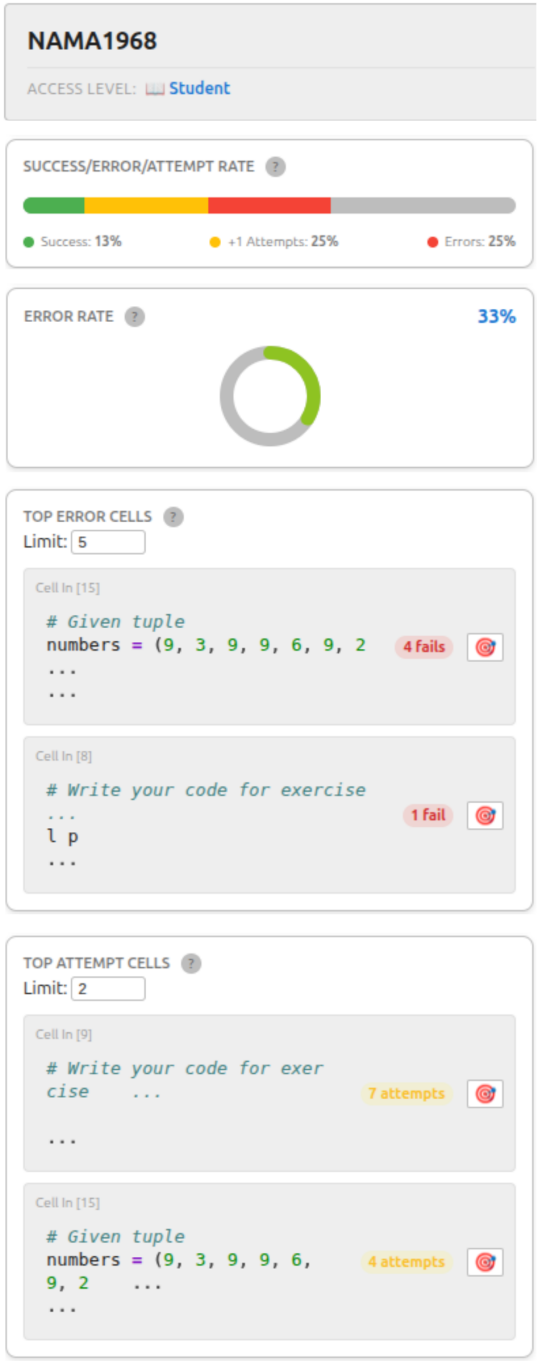


Figure 2: The upper section of the learner-facing JupyterLab dashboard.

Error Rate Displays overall execution failure rates using a dynamic color gradient scale (0% green to 100% red) according to Equation (2).

$$\text{Error Rate} = \frac{\text{Number of executions with errors}}{\text{Total number of executions}} \times 100\% \quad (2)$$

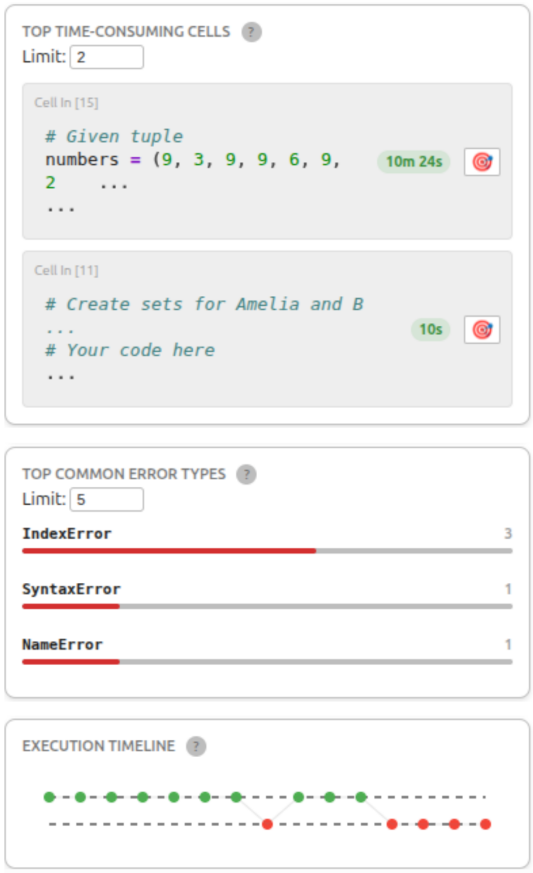


Figure 3: The lower section of the learner-facing JupyterLab dashboard.

Top Error Cells Lists the top n cells with the most failed executions, including a jump-to-cell navigation link.

Top Attempt Cells Highlights the top n cells requiring the highest volume of total executions.

Top Time-Consuming Cells Ranks cells by total time active during the workspace session.

Top Common Error Types Tallies occurrences of specific Python exceptions (e.g., `SyntaxError`, `NameError`).

Execution Timeline A chronologically plotted dot graph of executions, color-coded by outcome (green for success, red for errors) to illustrate workflow trends over time.

The dashboard metrics are intended as reflection cues rather than diagnostic measures of programming ability or learning. A high error rate or recurring error type may encourage students to inspect the corresponding messages and revisit the underlying concept. Repeated executions may prompt learners to review their current strategy or test their code in smaller steps, while time-consuming cells may indicate points where task decomposition or additional support could be helpful. However, these patterns can have multiple explanations such as repeated executions which also reflect productive experimentation, and long active times which represent deliberate work rather than difficulty. The *Help* function

Table 1: Descriptive Statistics of Main Evaluation Measures ($N = 22$)

Measure	M	SD	Min	Max
System Usability Scale (SUS)	65.57	17.89	40.00	90.00
TAM Overall	5.36	0.88	3.25	6.75
Perceived Usefulness (PU)	4.93	1.18	3.33	7.00
Perceived Ease of Use (PEOU)	5.78	1.06	3.17	7.00
NASA-TLX	8.15	2.92	1.67	12.67

therefore explains each metric, provides a cautious interpretation, and suggests possible next actions.

4 Evaluation

To evaluate how students experience the dashboard as a learner-facing reflection interface, we conducted an exploratory, cross-sectional user study. The aim was to assess students' perceptions of the dashboard in terms of usability, perceived usefulness, ease of use, and workload. Twenty-two (22) students participated in the study and interacted with the dashboard while solving Python exercises in JupyterLab. Participants differed in their prior experience with Jupyter Notebook and Python, allowing the dashboard to be evaluated by learners with varying programming backgrounds.

The study was approved by the Ethics Committee of the Faculty of Computer Science at the University of Duisburg-Essen[3]. After providing informed consent, participants first answered background questions on their previous experience with Jupyter Notebook and Python. They then used the JupyterLab dashboard during a programming task and subsequently completed standardized post-use questionnaires[4]. Usability was measured with the System Usability Scale (SUS), a ten-item instrument resulting in a score from 0 to 100 [5]. Perceived usefulness and perceived ease of use were assessed using items based on the Technology Acceptance Model (TAM) [4]. Subjective workload was measured with the NASA Task Load Index (NASA-TLX), covering mental demand, physical demand, temporal demand, perceived performance, effort, and frustration [7].

The collected data were analyzed descriptively to summarize students' perceptions of the dashboard. In addition, Spearman rank correlations were computed exploratively to examine relationships between usability, perceived usefulness, ease of use, and workload. This evaluation provides an initial empirical basis for understanding whether fine-grained notebook visualizations can be integrated into students' programming workflows without imposing excessive cognitive burden.

5 Results

The final sample consisted of 22 students with varying prior experience in Jupyter Notebook and Python. On average, participants spent about 17 minutes testing the dashboard while solving Python exercises. All multi-item scales showed acceptable to excellent internal consistency (SUS: $\alpha = .794$; PU: $\alpha = .852$; PEOU: $\alpha = .941$). Descriptive statistics are reported in Table 1.

[3]Ethics certificate ID: 2601CMPL2910.
[4]Questionnaires are available here.

The mean SUS score (65.57) fell just below the common benchmark of 68, indicating generally usable but improvable interface design; presenting multiple fine-grained indicators at once may have raised perceived complexity for some users.

Technology acceptance was positive overall (TAM $M = 5.36$ on a seven-point scale), but Perceived Ease of Use exceeded Perceived Usefulness (5.78 vs. 4.93). Students thus found the dashboard easy to operate, while its immediate value for self-reflection was less evident. Qualitative comments echoed this: some participants saw the dashboard as more useful for instructors, or asked for more actionable support such as guidance on resolving specific errors.

Subjective workload was low to moderate (NASA-TLX $M = 8.15$ on a 0–20 scale), suggesting the dashboard did not impose excessive cognitive burden. Together, these results indicate that fine-grained notebook visualizations can be integrated into students' workflows in a usable, low-workload manner, but that their reflective value depends on stronger interpretive and actionable support.

6 Conclusions

This study presented and evaluated a learner-facing JupyterLab dashboard that visualizes fine-grained notebook interactions to support students' reflection on their programming processes. The results suggest these visualizations can be integrated into notebook-based workflows with good usability and low workload: students found the dashboard relatively easy to use and did not report excessive cognitive burden, though they rated its usefulness for self-reflection more moderately. Visibility is thus only the first step. Learners also need support in interpreting the information and translating it into concrete next steps. This work contributes an initial learner-centered evaluation of fine-grained notebook visualizations as reflection interfaces in programming education. This exploratory study has several limitations. The evaluation involved 22 students who interacted with the dashboard for approximately 17 minutes. The cross-sectional post-use design captures participants' perceived usability, usefulness, ease of use, and workload after short-term use, but it does not establish whether the dashboard improves self-reflection, debugging behavior, programming strategies, or learning outcomes. The findings should therefore be interpreted as initial evidence of short-term user experience rather than pedagogical effectiveness. Future longitudinal or controlled studies should examine how students use the dashboard over time and whether it leads to changes in programming behavior and learning.

References

[1] Zhenyu Cai, Richard Lee Davis, Raphaël Mariétan, Roland Tormey, and Pierre Dillenbourg. 2025. Jupyter Analytics: A Toolkit for Collecting, Analyzing, and Visualizing Distributed Student Activity in Jupyter Notebooks. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS 2025)*. ACM, Pittsburgh, PA, USA, 1–7. doi:10.1145/3641554.3701971

[2] Nuo Cheng, Wei Zhao, Xiaoqing Xu, Hongxia Liu, and Jinhong Tao. 2024. The influence of learning analytics dashboard information design on cognitive load and performance. *Education and Information Technologies* 29, 15 (April 2024), 19729–19752. doi:10.1007/s10639-024-12606-1

[3] Linda Corrin and Paula de Barba. 2014. Exploring students' interpretation of feedback delivered through learning analytics dashboards. In *Proceedings of the 31st ASCILITE International Conference on Innovation, Practice and Research in Use of Educational Technologies in Tertiary Education.* Australasian Society for Computers in Learning in Tertiary Education (ASCILITE), Dunedin, New Zealand.

[4] Fred D Davis et al. 1989. Technology acceptance model: TAM. *Al-Suqri, MN, Al-Aufi, AS: Information seeking behavior and technology adoption* 205, 219 (1989), 5.

[5] German UPA. 2026. System Usability Scale (SUS). https://germanupa.de/wissen/fragebogenmatrix/sus Accessed: 2026-02-22.

[6] Iris Groher, Michael Vierhauser, and Erik Hartl. 2024. A Learning Analytics Dashboard for Improved Learning Outcomes and Diversity in Programming Classes. In *Proceedings of the 16th International Conference on Computer Supported Education - Volume 2: CSEDU*. INSTICC, SciTePress, Angers, France, 618–625. doi:10.5220/0012735000003693

[7] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, Amsterdam, The Netherlands, 139–183. doi:10.1016/S0166-4115(08)62386-9

[8] Ioana Jivet, Maren Scheffel, Marcel Schmitz, Stefan Robbers, Marcus Specht, and Hendrik Drachsler. 2020. From students with love: An empirical study on learner goals, self-regulated learning and sense-making of learning analytics in higher education. *The Internet and Higher Education* 47 (2020), 100758. doi:10.1016/j.iheduc.2020.100758

[9] Lucas Paulsen and Euan Lindsay. 2024. Learning analytics dashboards are increasingly becoming about learning and not just analytics-A systematic review. *Education and Information Technologies* 29, 11 (2024), 14279–14308.

[10] Jeffrey M Perkel. 2018. Why Jupyter is data scientists' computational notebook of choice. *Nature* 563, 7732 (2018), 145–147.

[11] Project Jupyter. 2024. *JupyterLab Extension Tutorial*. Project Jupyter. https://jupyterlab.readthedocs.io/en/v4.5.x/extension/extension_tutorial.html Accessed: 2026-02-22.

[12] Project Jupyter. 2024. jupyterlab/extension-template: A cookiecutter template for creating a JupyterLab extension. https://github.com/jupyterlab/extension-template.

[13] Deepthi Raghunandan, Aayushi Roy, Shenzhi Shi, Niklas Elmqvist, and Leilani Battle. 2023. Code Code Evolution: Understanding How People Change Data Science Notebooks Over Time. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 863, 12 pages. doi:10.1145/3544548.3580997

[14] Derek Robinson, Neil A. Ernst, Enrique Larios Vargas, and Margaret-Anne D. Storey. 2022. Error identification strategies for Python Jupyter notebooks. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension (ICPC '22)*. ACM, New York, NY, USA, 253–263. doi:10.1145/3524610.3529156

[15] Beat A Schwendimann, Maria Jesus Rodriguez-Triana, Andrii Vozniuk, Luis P Prieto, Mina Shirvani Boroujeni, Adrian Holzer, Denis Gillet, and Pierre Dillenbourg. 2016. Perceiving learning at a glance: A systematic literature review of learning dashboard research. *IEEE transactions on learning technologies* 10, 1 (2016), 30–41.

[16] Jigneshkumar Jitubhai Sondagar. 2021. *E-Learning - Analytics using Jupyter Notebooks*. Bachelor Thesis. Hochschule Merseburg (FH), Merseburg, Germany.

[17] Manuel Valle Torre, Thom van der Velden, Marcus Specht, and Catharine Oertel. 2025. *JELAI: Integrating AI and Learning Analytics in Jupyter Notebooks*. Springer Nature Switzerland, Cham, Switzerland, 68–75. doi:10.1007/978-3-031-98465-5_9